

MODULE - 3

Advanced Heap Structures

> Heap $\rightarrow$ Binary Heap

Heap is a special type of binary tree which satisfies the following 2 properties:-

① Structure property:- It is a 'complete binary tree', meaning all levels have maximum number of nodes, except possibly the last level, which is filled from left to right.

② Heap-order property:- The key value in any Node, 'N' is related to the values of its children, depending on the type of heap.

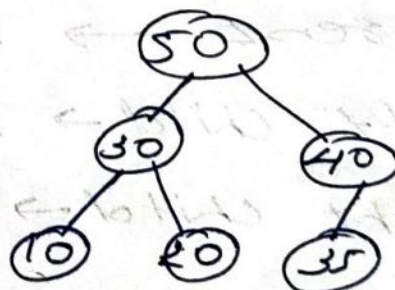
> Types of heaps

(a) Max Heap

In Max heap, the value of each node is greater than or equal to the value of its children.

Therefore, the largest element always at the root.

Example :-

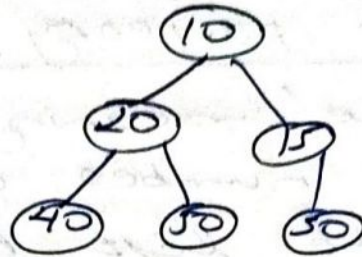


(b) Min Heap

• In a Min Heap, the value of each node is less than or equal to the value of its child.
Hence, the smallest element is always at the root.

Ex :-

~~Index:~~



> Applications of Heap

- ① Implementing Priority queue
- ② Heap sort algorithm
- ③ Graph algorithms
- ④ Job scheduling systems.

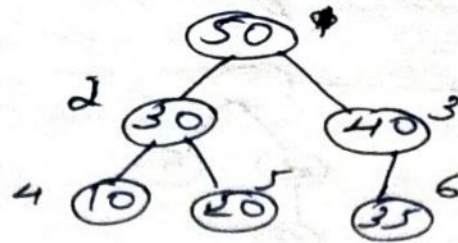
> Representation of Heap

A heap is usually represented using an array, as it is a complete binary tree.

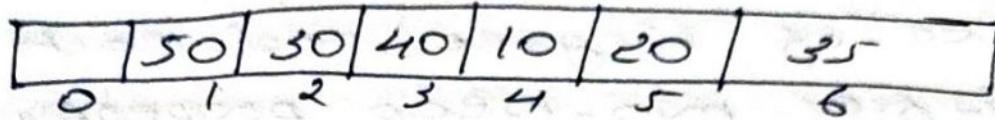
• If the root is stored at index i :-
then for a node at index i :-

- Parent $\rightarrow i/2$
- Left child $\rightarrow 2*i$
- Right child $\rightarrow 2*i+1$

EXAMPLE :- Consider a max heap:-



Array representation is:-



> BINOMIAL HEAP

A binomial heap is a collection of binomial trees that together satisfy the heap property.

• It is an advanced heap data structure that supports efficient merge (union) operations.

Binomial heap = a set of binomial trees arranged in increasing order of degree, each obeying the min-heap or max-heap property.

> Binomial Tree

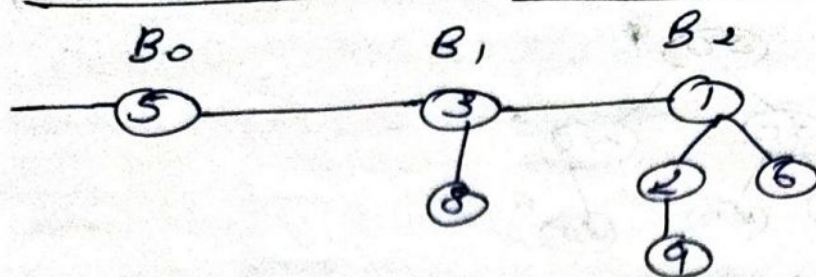
A binomial tree, B_k , is defined recursively as:-

- B_0 - a single node
- B_k - formed by linking two B_{k-1} trees. one becomes the leftmost child of the other's root.

> Properties

- Number of nodes = 2^k
- Height = Root degree = k

Example: - Min Binomial Heap



All roots (5, 3, 1) maintain in order of degree, and each tree satisfies min-heap property.

> Basic Operations on Binomial Heap

> ① Union (H, H_2)

② Insert (H, x)

③ Extract-min (H)

> ④ Decreasing a key

The Decrease-key() operation is used to decrease the value of a key in a binomial heap.

Once a key's ~~heap~~ value is reduced, it may become smaller than the key of its parent, which constitutes a violation of min-heap property. In such cases, swap the element with its parent, grandparent and so forth until the min-heap property is attained.

> Algorithms

BINOMIAL-HEAP-DECREASE-KEY(H, x, k)

- 1) If $k > \text{key}[x]$
- 2) then error "new key is greater than current key"
- 3) $\text{key}[x] \leftarrow k$
- 4) $y \leftarrow x$
- 5) $z \leftarrow p[y]$
- 6) While $z \neq \text{NIL}$ and $\text{key}[y] < \text{key}[z]$
- 7) do exchange $\text{key}[y] \leftarrow \text{key}[z]$
- 8) If y and z have satellite fields exchange them too
- 9) $y \leftarrow z$
- 10) $z \leftarrow p[y]$

Explanation

Firstly, we need to check whether the given key is less than the origin value.

ex:- If node has value 45.

a) key, $k = 48$

$48 > 45$

↓
Error

b) $k = 7$

$7 < 45$

↓
Continue

- (i) Firstly, assign the new key value to the node
- (ii) $y \rightarrow \text{node}$, $z \rightarrow \text{parent of } y$
- (iii) Loop :- until root is reached ($z = \text{NIL}$) and whenever $\text{key}[y] < \text{key}[z]$ → exchange $\therefore$ Min-heap property

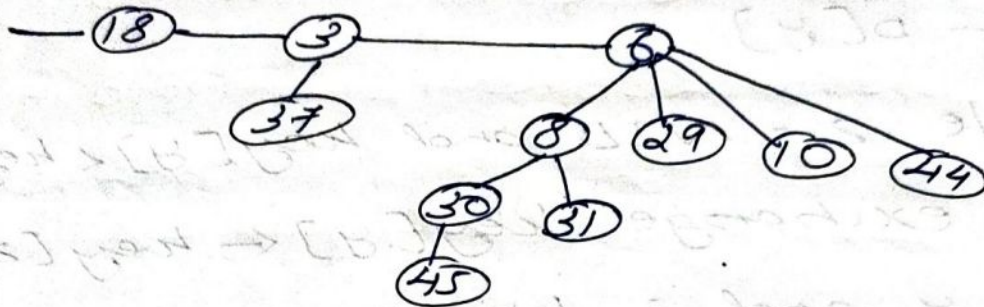
satellite values means addition
fields associated with it.
Then, next iteration,

$$y \leftarrow 2$$

$$2 \leftarrow P(y)$$

> Example

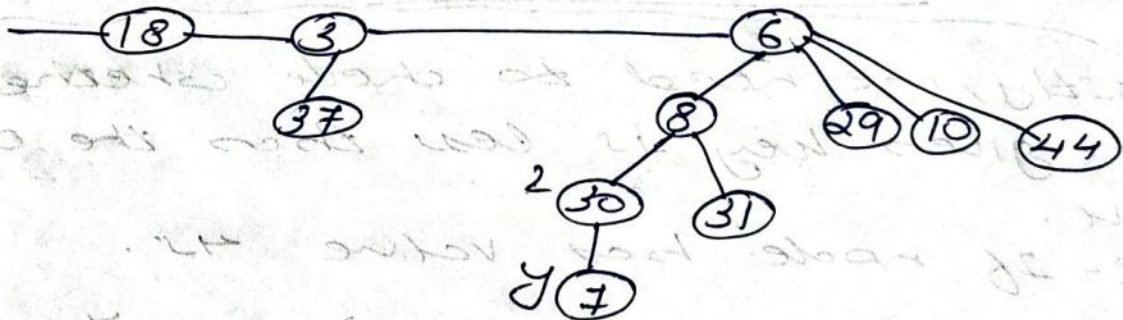
consider the following binomial he



Decrease the node 45 to 7.

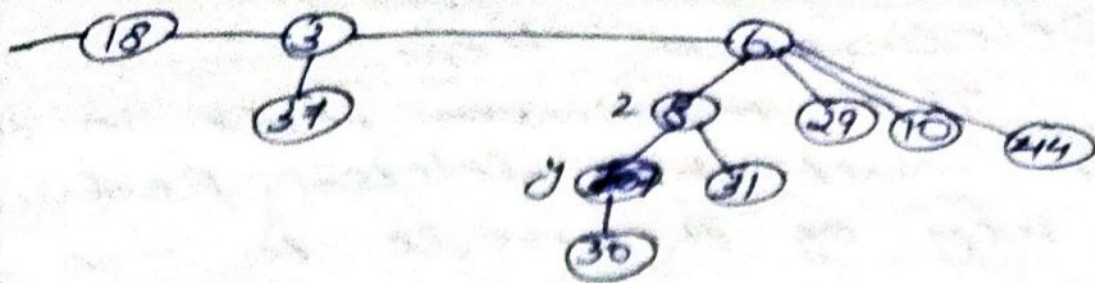
Ans:- Since, $7 < 45$, we can proceed

(i) Firstly exchange the value of
node 45 with 7.



(ii) Now, $y = 7$ and $z = 30$

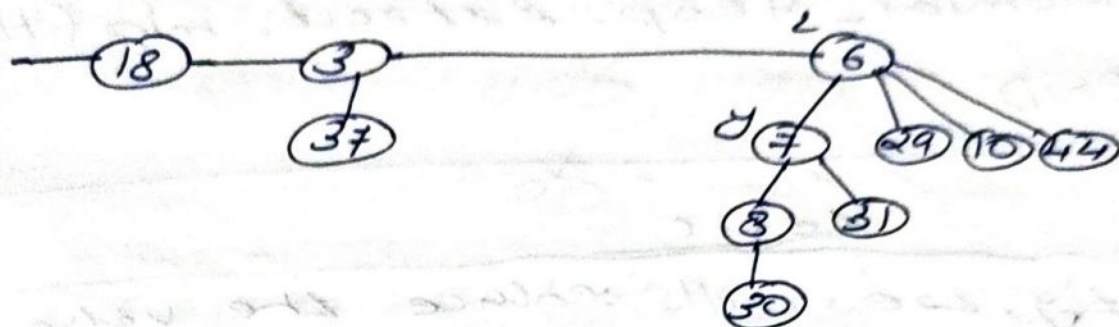
$7 < 30$, i.e., $y < z$. So, min-heap
property violated. So, exchange
7 and 30.



(ii) Now, $y \leftarrow 7$, $z \leftarrow 8$

$7 < 8$, i.e. , $y < z$. So, min-heap property violated.

So, exchange 7 and 8.



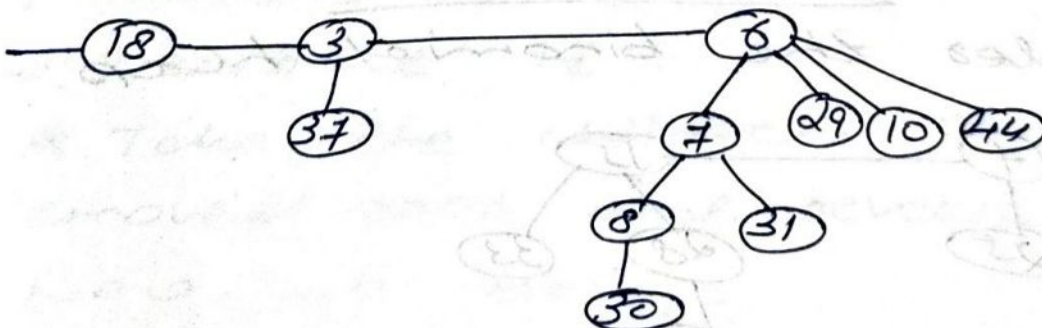
(iv) Next, $y \leftarrow 7$ and $z \leftarrow 6$

Next, $7 > 6$, i.e. , $y > z$ [Min-heap property satisfied]
So, no exchange needed.

(v) Now, $y \leftarrow 6$ and $z \leftarrow \text{NIL}$

So, END

$\therefore$ Our final binomial heap will be



⑤ Deleting a key

The minimum node in the heap must be deleted. Reduce the key of the node to $-\infty$ for

Algorithm

Binomial-Heap-Delete(H, x)

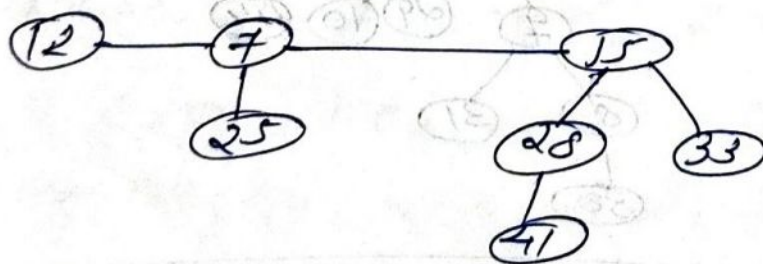
- ① Binomial-Heap-Decrease-key(H, x)
- ② Binomial-Heap-Extract-min(H)
- ③ END

Logic

Firstly, we will reduce the value of the node to be deleted to $-\infty$. Then it will become a root node of one of the trees. Then, we will extract min (since, $-\infty$, is the smallest, it will be extracted). Then after deleting it, merging children are re-merged.

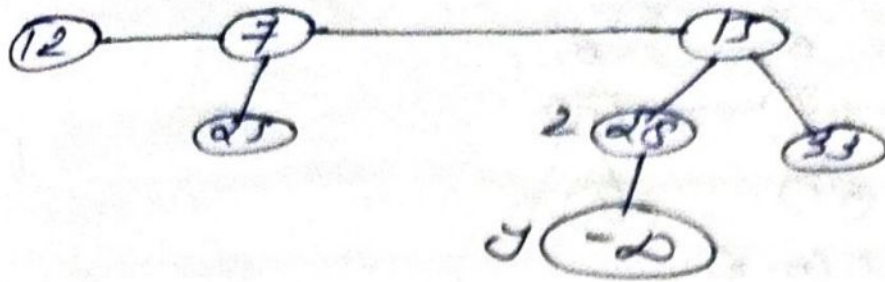
Example

- ① Consider the binomial heap:-

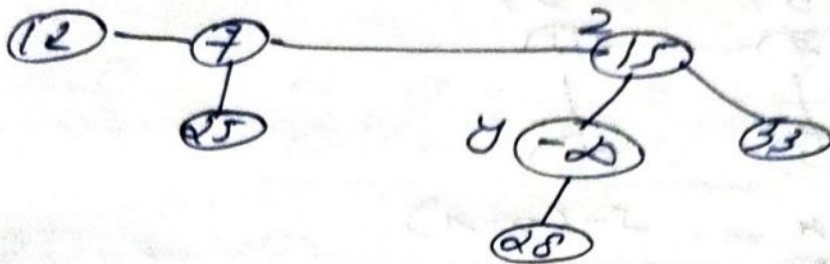


Delete the node 41 from the given binomial heap.

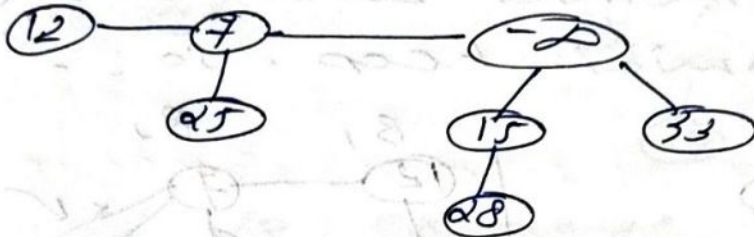
Ans: Step 1 :- Assign the value of 41 to -20



$y < 2$, so, exchange y and 2



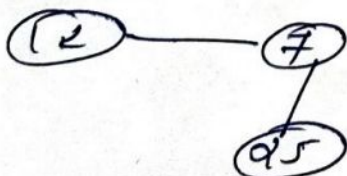
$y < 2$, so, exchange y and 2



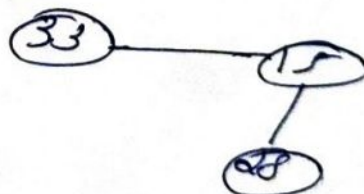
Step 2 :- Extract min from the binomial heap

Minimum value among roots = -20
So, extract it.

Take the children of the removed root and reverse their order.
Now, we have:-



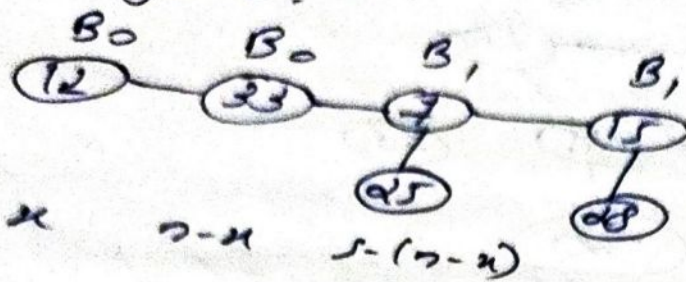
+



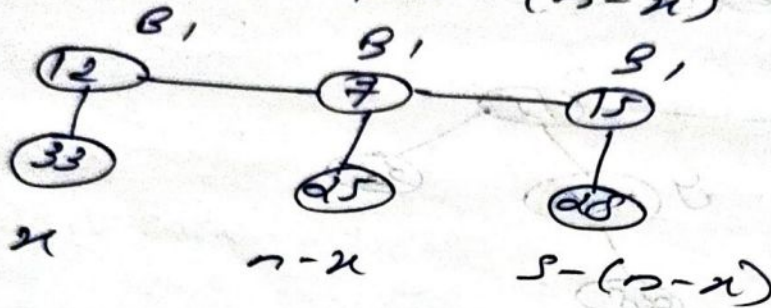
Reverse
order
children
of
-20

union

Perform union operation
Arrange in ascending order:
degree: -

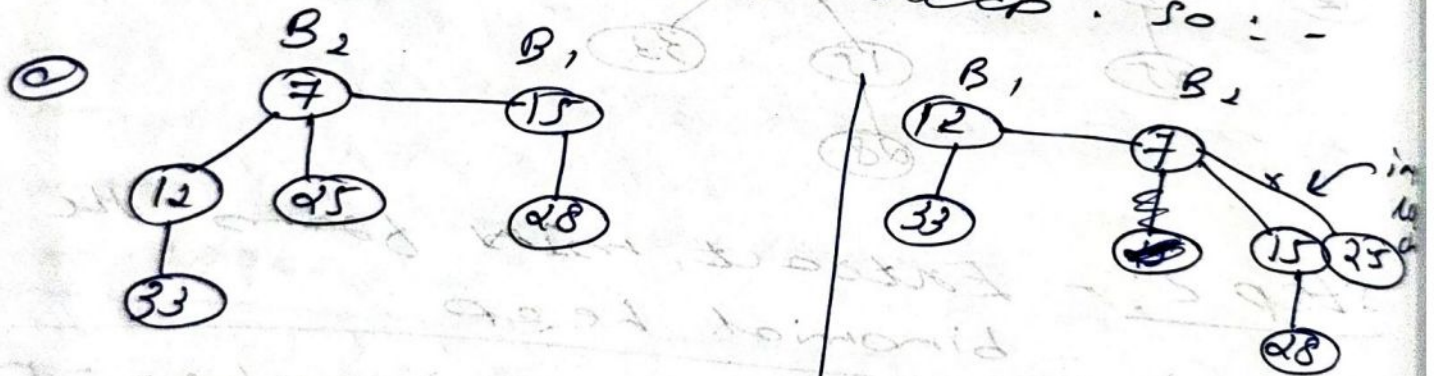


$x = n-x \neq s-(n-x) \rightarrow \text{Merge}$

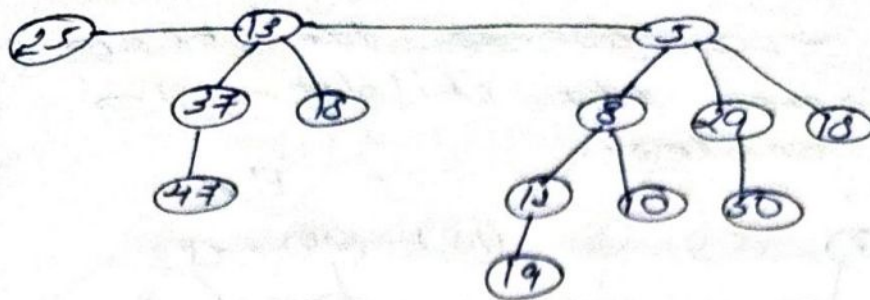


Here, $x = n-x = s-(n-x) \rightarrow \text{shift}$

No two binomial trees can exist in a binomial heap. So: -

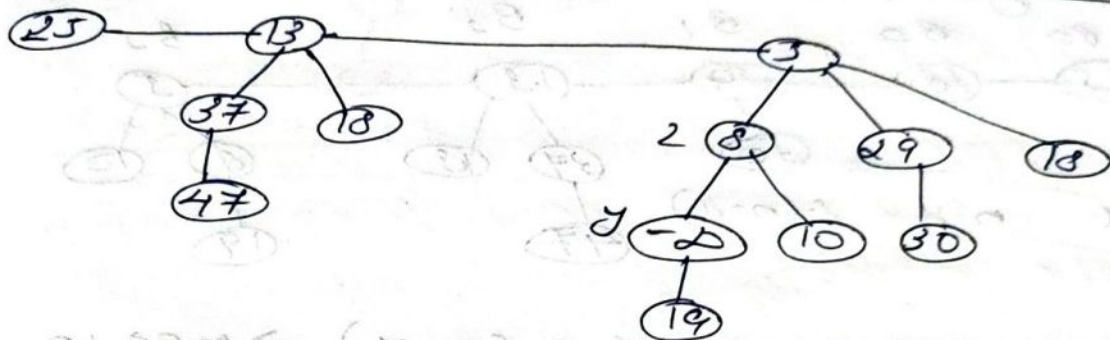


② Consider the binomial heap:-

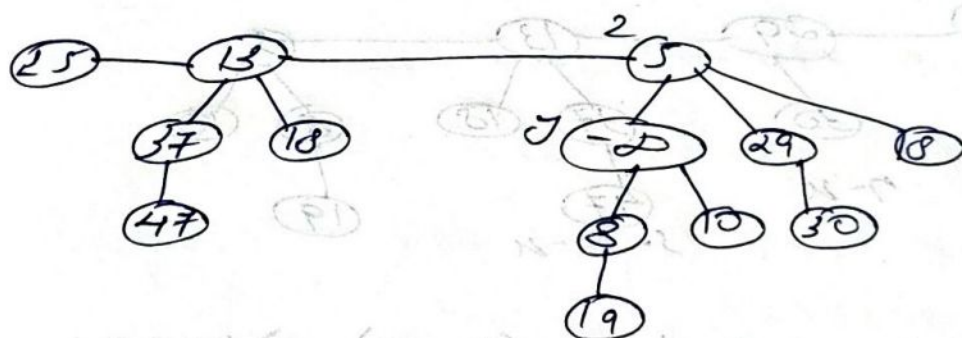


delete the key 12 from the given binomial tree.

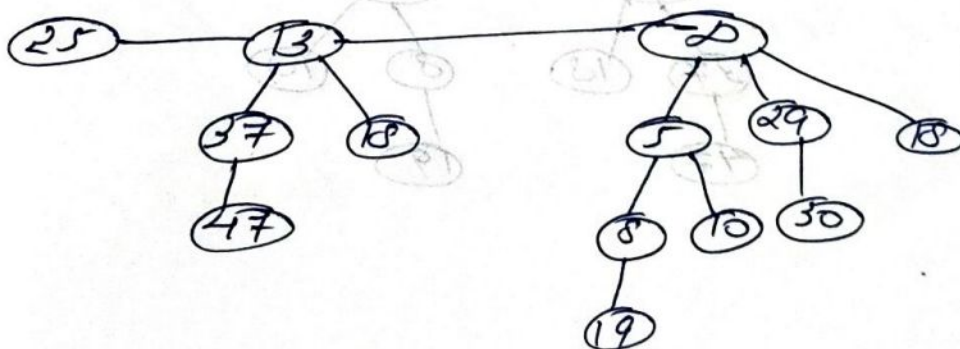
Ans:- Step 1:- Assign the value of 12 with $-\infty$



$y < 2$, so, exchange

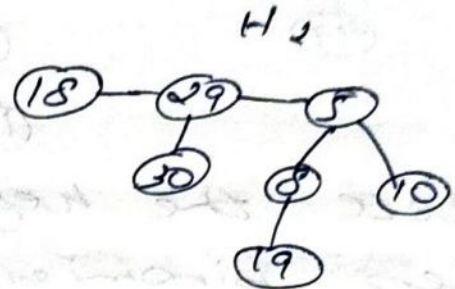
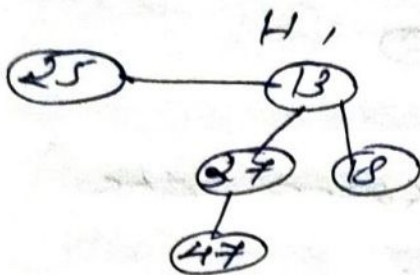


$y < 2$. Exchange



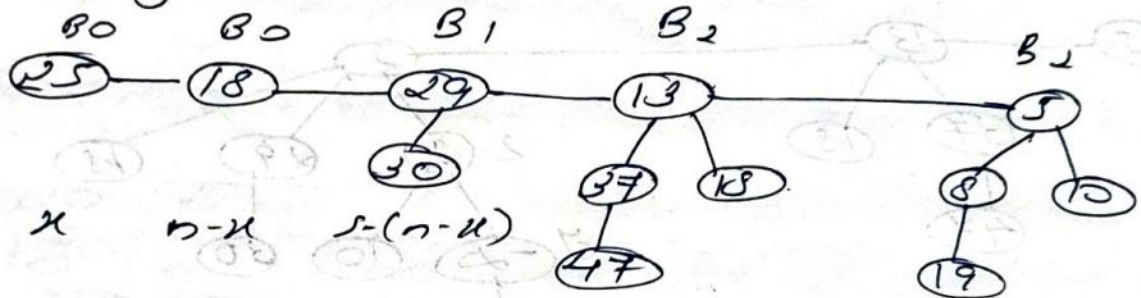
Step 2 :- Extract MIN

Remove $\rightarrow$ from the heap and arrange its children in reverse order.

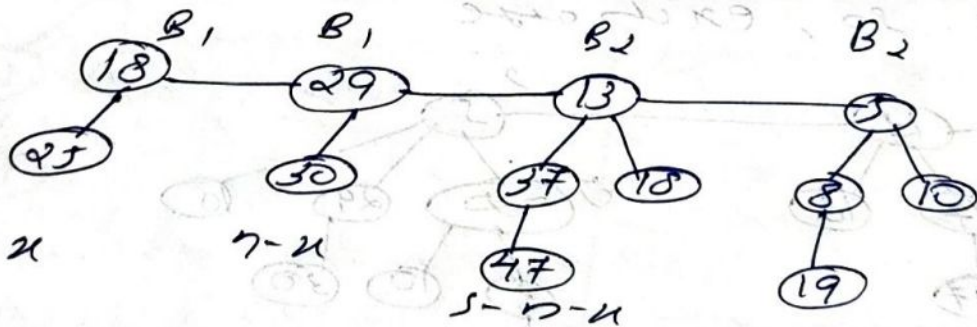


• Union (H_1, H_2)

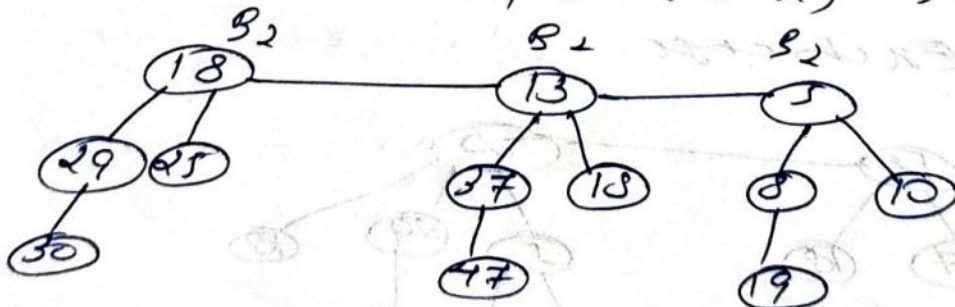
* Arrange in ascending order of degree.



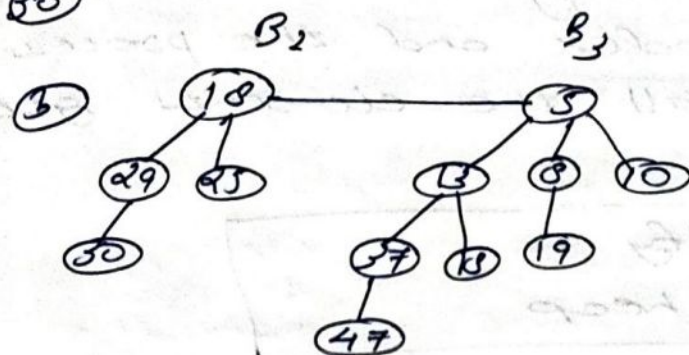
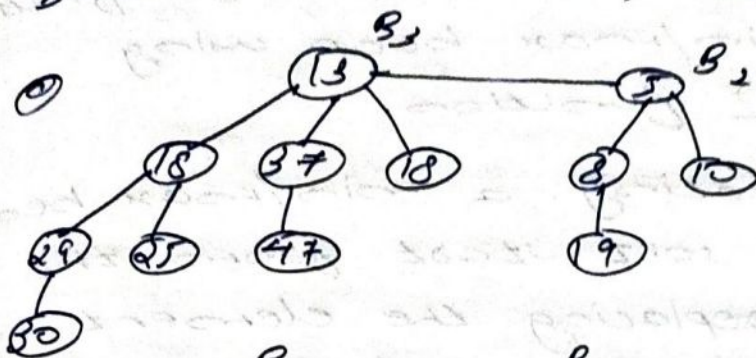
Here, $x = n-x \neq s-(n-x) \rightarrow$ merge



Here, $x = n-x \neq s-(n-x) \rightarrow$ merge



No two binomial trees of the same degree can exist in a binomial heap. To, merge them.



Problem

Convert the following elements into a heap-sorted order.

4, 10, 3, 2, 1

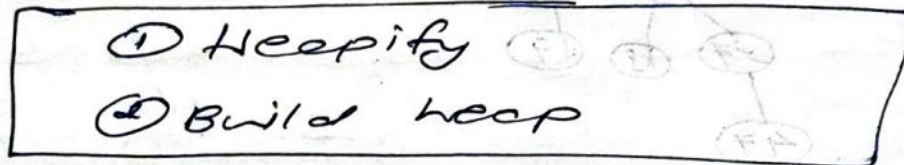
30/10/2025
Thursday

HEAP SORT

Heap sort consists of 2 functions.

① Build min/max heap using heapify() function.

② After creating a min/max heap the heap sort starts from the root by replacing the element in the last index and the process continues till the elements are sorted.



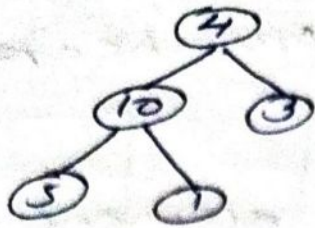
> Steps

- ① Build a min heap / max heap for the array.
- ② Swap the root with the last element.
- ③ Reduce heap size by 1.
- ④ Heapify again the root.
- ⑤ Repeat until all the elements are sorted.

Problem

- ① Construct the following elements
4, 10, 3, 5, 1
into a heap-sorted order.

Ans: Step 1 :- Construct a heap



Step 2 :- Sort

Firstly, check whether the formed heap is min heap $\rightarrow$ No.

Consider the lower branches:-



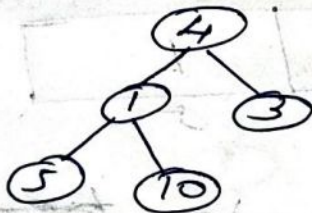
} Not a min-heap

Replace 1 with 10.



} min-heap

Now tree becomes:-

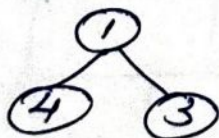


Compare branches:-



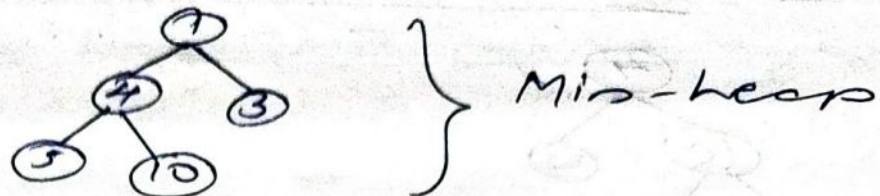
} Not a min-heap

Replace 1 with 4

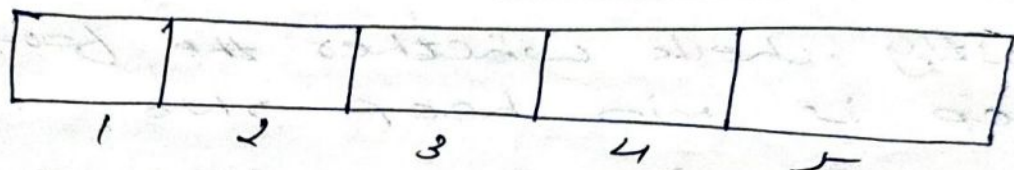


} min heap

Now tree becomes:-

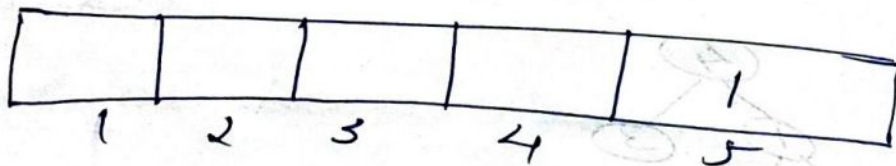
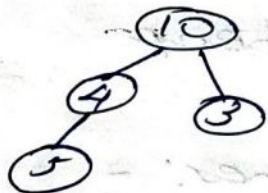


Since there are 5 elements,
construct an array as follows:

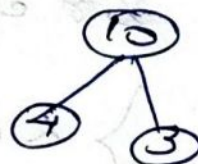


Here, move the root element to the last ~~ele~~ index of the array.

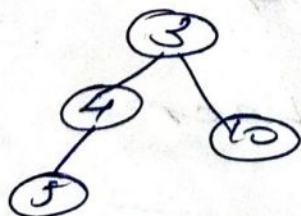
Now replace root 1 with the last element 10 and reduce ~~s~~ by 1.



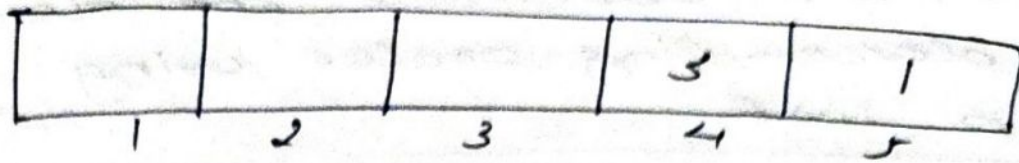
The tree is not in ~~tree~~ min-heap



Now, tree becomes:-



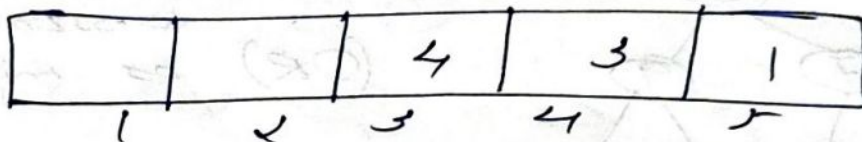
- Move 3 to array and exchange 3 with the last element 1 and reduce heap size by 1.



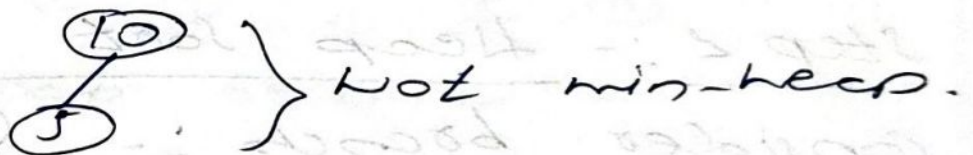
- Replace 4 and 5 :-



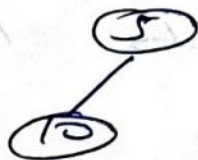
- Move 4 to array



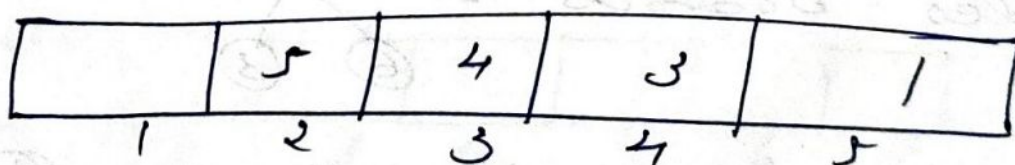
- Exchange root 4 with last element 10 and reduce heap size by 1.



- Interchange 5 and 10 :-



- Move 5 to array



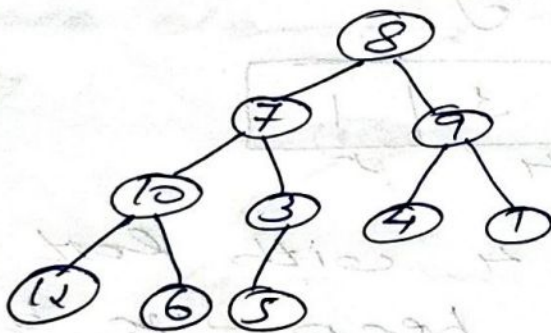
- Exchange 5 and 10 and reduce heap size by 1. Now we only have 10. Add it to array.

10	5	4	3	1
1	2	3	4	5

Now, we obtained the elements in descending order using heap sort.

② Sort the following elements
8, 7, 9, 10, 3, 4, 1, 12, 6, 5
using heap sort [min-heap]

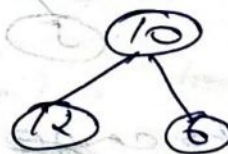
Ans:- Step 1:- Construct a heap



while constructing
construct it
(OR) as min-heap

Step 2:- Heap sort

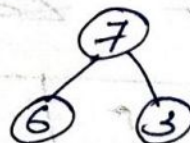
Consider branch :-



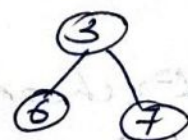
Exchange 6 with 10 $\Rightarrow$



Consider branch :-



Exchange 3 with 7 $\Rightarrow$



Consider branch :-



Exchange 1 with 9 $\Rightarrow$



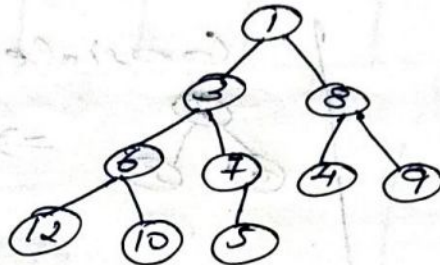
Consider branch :-



Exchange 1 with 8 $\Rightarrow$



Now the ~~tree~~ ^{heap} becomes :-



Consider



$\Rightarrow$



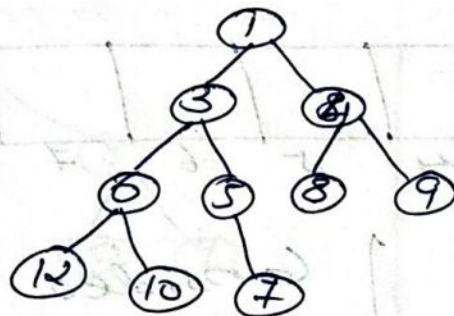
Consider



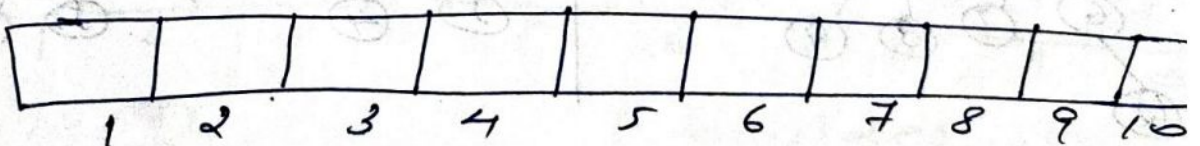
$\Rightarrow$



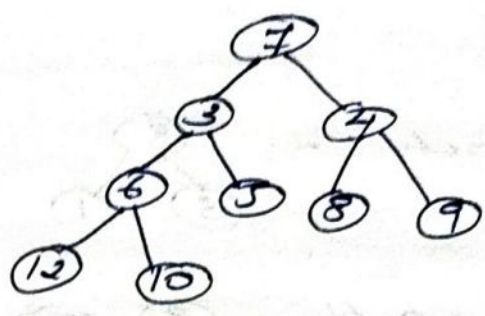
Now heap becomes :-



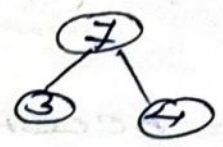
Create an array :-



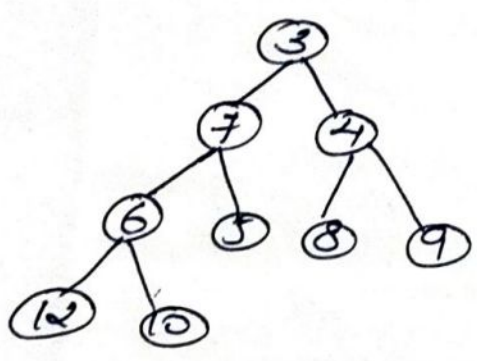
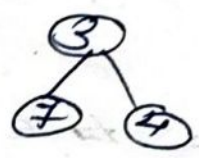
Move the root element to the 1 index of the array. Now replace root with last element 7 & re



Consider



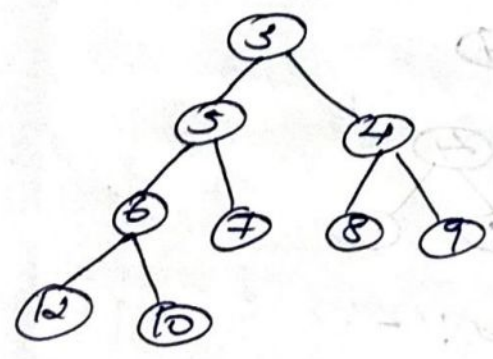
⇒



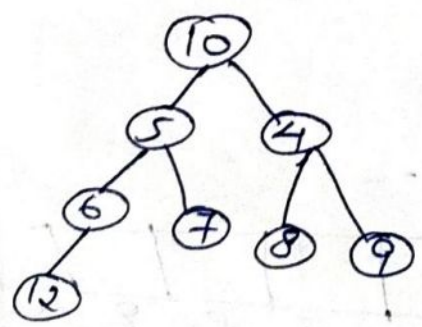
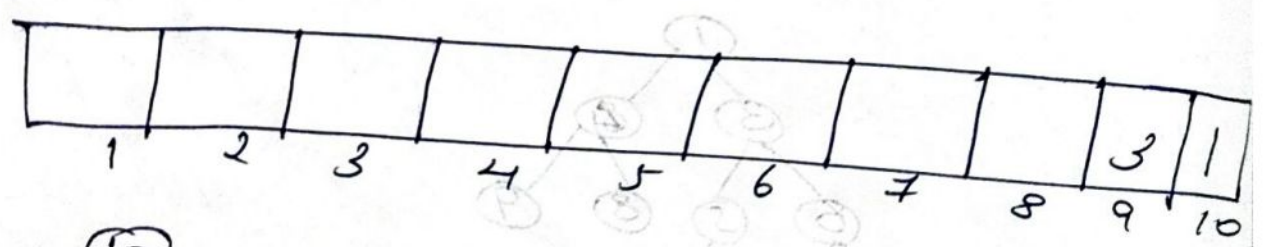
Consider



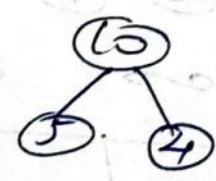
⇒



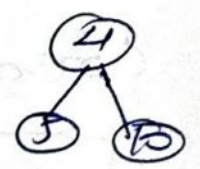
Move 3 to array
Replace 3 with

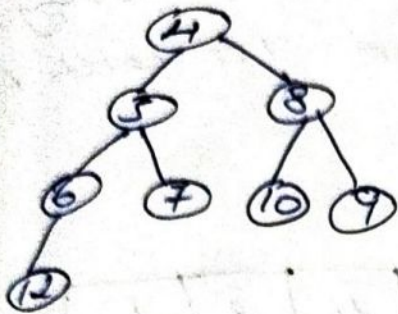
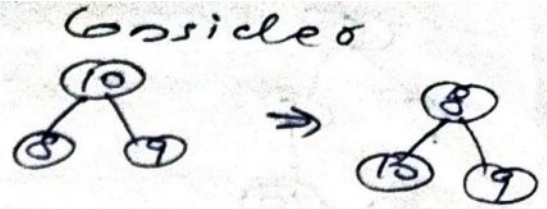
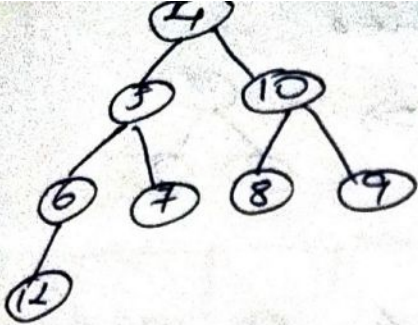


Consider

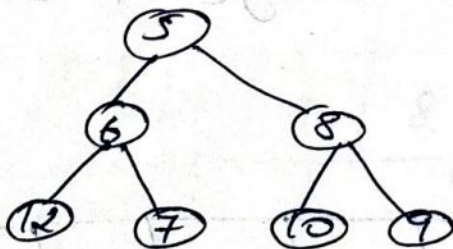
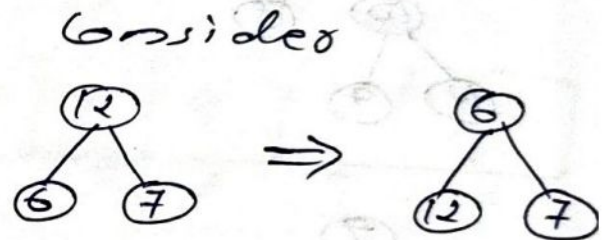
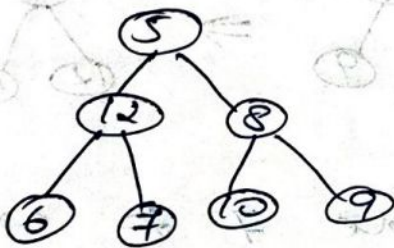
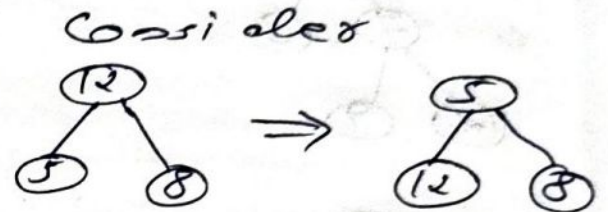
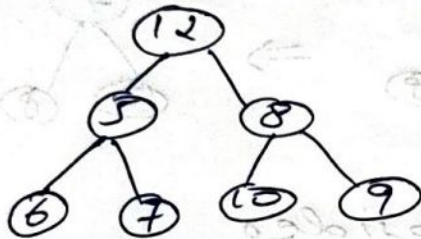
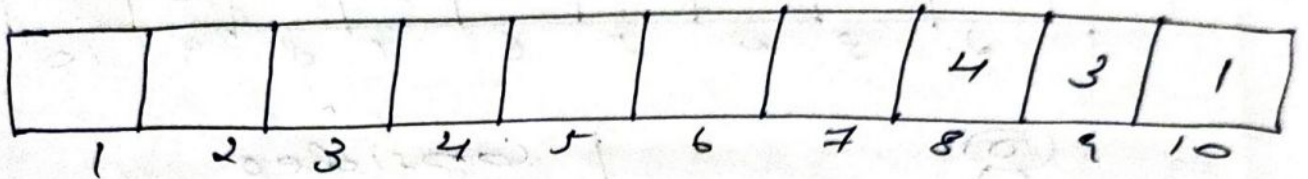


⇒

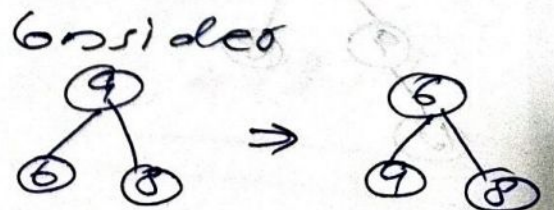
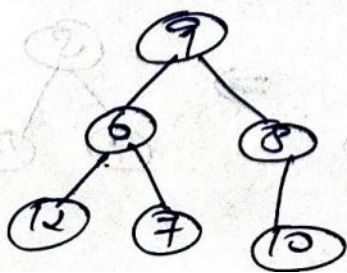
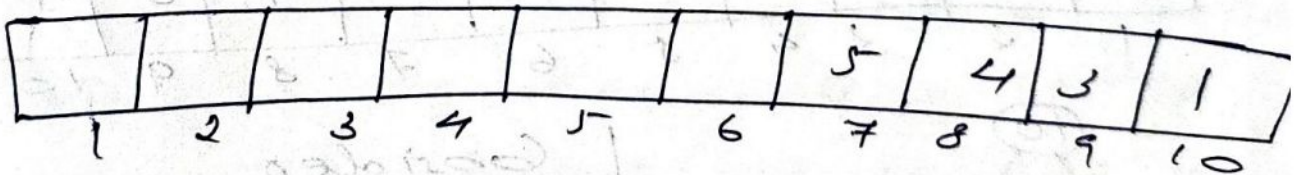


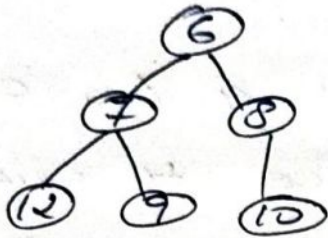
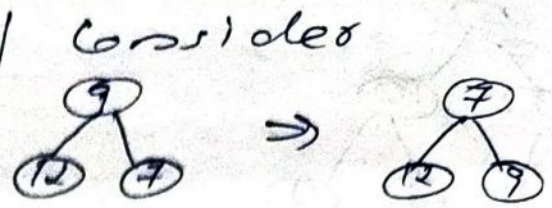
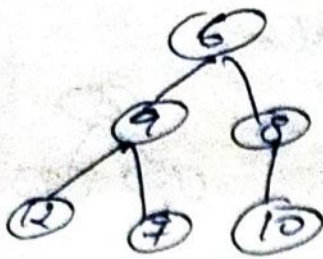


Move 4 to array
Replace 4 with 12

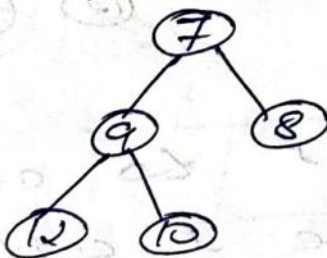
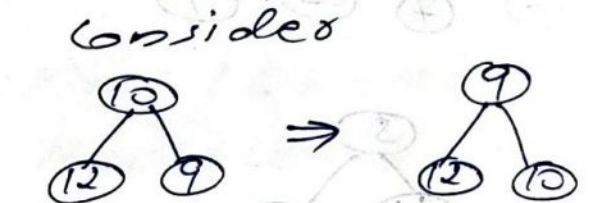
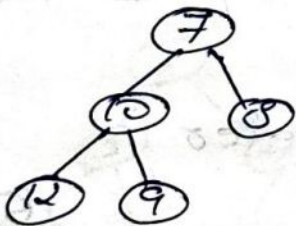
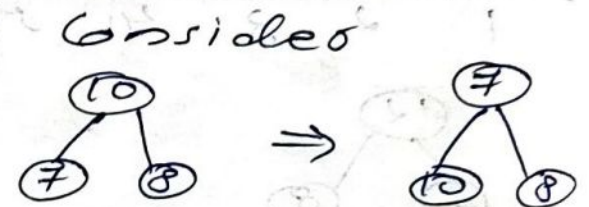
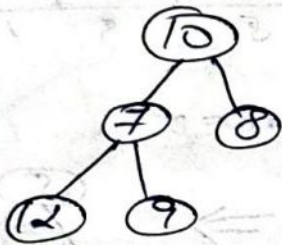
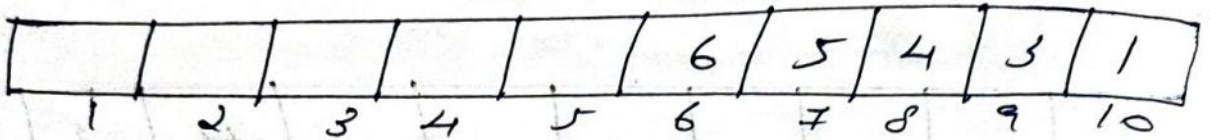


Move 5 to array
Replace 5 with

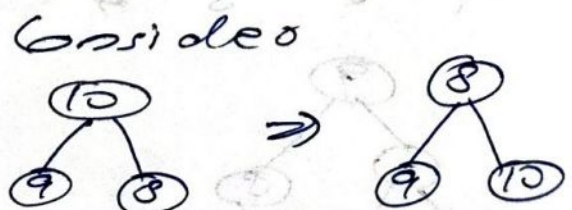
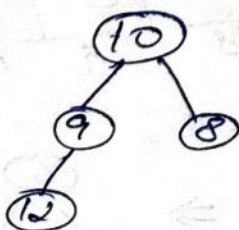
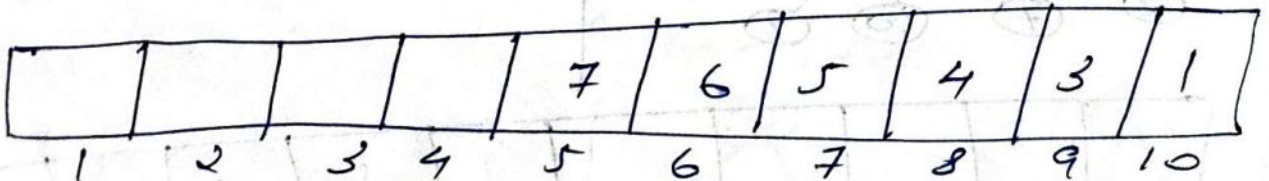


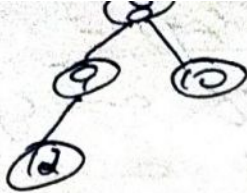


Move 6 to array
Exchange 10 with

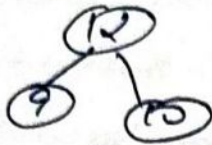
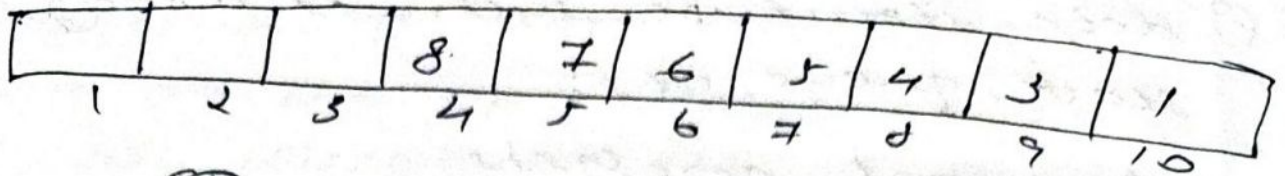


Move 7 to array
Exchange 7 with

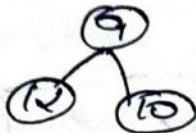




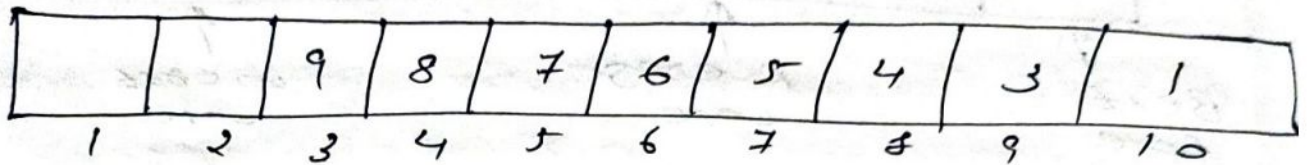
Move 8 to array.
Exchange 12 with 9.



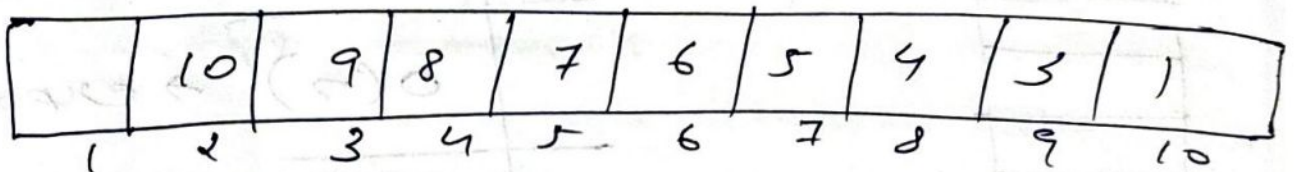
Exchange 9 with 12



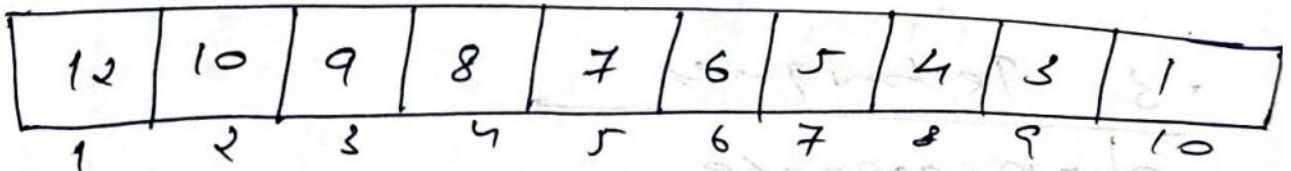
Move 9 to array.
Exchange 10 with 8.



Move 10 to array.



Move 12 to array.



Hence, we sort the heap root elements in descending order using min-heap as follows:-

12, 10, 9, 8, 7, 6, 5, 4, 3, 1